

StarryOS 自举编译与内核改进成果汇报

BigLab-B 最终汇报 | 杨凯森 | 2026-05

主线：让 StarryOS 在 guest 内编译出 StarryOS 自己，过程分为两步推进。

1

首先实现单核

先让 StarryOS 在 guest 环境中稳定完成一次自举编译，形成单核可复现基线。

2

然后实现多核编译

再沿同一条链路推进多核，让 StarryOS 在 guest 内以多核方式完成自举编译。

BigLab-A: rCore Tutorial 章节训练

基于 yks23/tg-rcore-tutorial, 完成 ch1/ch2 基础、ch3-ch6 内核机制、ch8 综合 demo, 以及 ch3-ch5 的 t2l10 扩展版本。

ch1-ch2: 基础内核

tg-rcore-tutorial-ch{1,2}-yks23

裸机启动、no_std/Rust、SBI/QEMU 运行; 用户程序加载、trap/syscall 和批处理执行。

ch3 - ch6: 核心 OS 机制

tg-rcore-tutorial-ch3/4/5/6-yks23

覆盖多道程序与时间片、地址空间与页表、进程
fork/exec/wait、文件系统与块设备。

ch8 + t2l10: 综合与扩展

tg-rcore-tutorial-ch8-yks23
ch{3,4,5}-yks23-t2l10

继续补线程/同步、图形或综合 demo, 并对前几章做扩展版本。

形成的能力

把 RISC-V、QEMU、trap、地址空间、进程、文件系统和同步这些基础机制跑通, 为后续 StarryOS/TGOSKit 的真实 workload 修复打底。

BigLab-A 的作用: 不是泛泛完成基础实验, 而是沿 rCore 章节把小内核的关键子系统逐步实现和验证。

BigLab-B Task 1: tg-arceos-tutorial 基础练习

Task 1 是基于 rcore-os/tg-arceos-tutorial 的 test 分支，在自己的 fork 中完成 5 个规定的 exercise-* 基础练习。

仓库与分支

fork 上游
tg-arceos-tutorial/test，到自己的仓库继续完成代码和总结文档。

本地对应仓库：github.com/yks23/tg-arceos-tutorial

完成的 5 个 exercise

exercise-printcolor
exercise-hashmap
exercise-altalloc
exercise-ramfs-rename
exercise-sysmap

训练到的能力

从基础输出、集合/哈希表、替代 allocator，到 RAMFS rename、系统符号/地址映射，逐步补齐 ArceOS 小实验能力。

- 这一部分不是 Harness，也不是 StarryOS PR；它是 BigLab-B 的基础训练任务。
- 完成后才进入 Task 2：把方法迁移到 TGOSKit / StarryOS 的真实内核改进。
- 后面的 Harness、PR、自举编译和多核测速都属于 Task 2 主线。

BigLab-B Task 2 阶段 1: Harness 初版

进入 Task 2 后，第一阶段尝试让模型生成测试并自动运行，用测试失败来反推 StarryOS 的 bug。

做法

- 围绕 syscall、busybox 小命令和文件系统行为生成测试用例。
- 模型读日志、提出怀疑点、继续补 case 或定位代码。

经验

- 能快速覆盖很多边界输入，但缺乏 workload 方向性。
- 真实 OS bug 数量不稳定，容易陷入泛化测试和低价值失败。

结论：纯测试挖掘可以作为补充，但不足以支撑大实验主线；后续改为真实 workload 驱动。

BigLab-B Task 2 阶段 1: Harness 当前闭环

Harness 后来从泛化测试挖掘，收敛到和 StarryOS dev 紧密联动：跑真实任务、读输出、修内核、补测例、提 PR。



自动同步

每天早上 9:00 拉取最新 dev，专门同步分支，减少上游快速演进带来的冲突成本。

自动修复闭环

在 StarryOS 上跑具体任务；一旦发现相关 bug，整理 root cause 与 Test Suite，随 OS 改动提 PR 到 dev。

PR 守护

定时检查 PR CI 状态；未通过则继续收集失败、修复、推送，直到达到可合并状态。

Task 2 PR: 12 个已合入 OS 修复

这些 PR 不按“改了多少文件”讲，而按 OS 行为边界拆分：进程线程、文件系统、网络 ABI、SMP 同步和调度前进性。

进程 / 线程 / futex

- #692 robust futex cleanup
- #693 vfork parent blocking
- #878 teardown context

文件系统 / 设备

- #695 rsex4 inode bitmap
- #800 device full transfer
- #844 tmpfs rename exec

SMP / 同步 / 调度

- #842 CPU topology
- #879 raw mutex handoff
- #885 syscall entry snapshot
- #926 SMP wakeup progress

网络 ABI

- #694 IPv4-mapped IPv6 socket

保持 IPv6 用户态语义，内部复用 IPv4 backend。

验证标准

每个 PR 均包含问题根因、修复范围、测试用例和 CI 或本地复现证据。

背景 1: 进程创建语义: fork / vfork / clone

启动新程序不是单个动作，而是创建子执行流、决定资源共享边界、替换程序映像、回收退出状态的一组 ABI。

术语定义

fork 复制当前进程得到子进程；vfork 是父进程阻塞的创建后立即 exec/exit 路径；clone 是用 flags 表达资源共享的底层创建接口。

fork: 独立子进程

文件描述符、信号等状态按规则继承；地址空间按复制或 COW 语义分离，父子随后并发运行。

vfork / CLONE_VFORK

子进程 exec 或 exit
前父进程必须等待，避免共享或尚未完全分离的状态被并发破坏。

clone: 底层 flag 组合

通过 flags 选择是否共享
VM、文件表、信号处理、线程组等资源；线程、fork、vfork 都可表达为不同组合。

exec / wait / posix_spawn

exec 替换子进程映像，wait 回收退出状态。shell、BusyBox、Cargo 会把这些动作串成高频启动链路。

进程创建正确性取决于资源共享边界、父子同步，以及 exec/wait 顺序是否与 Linux ABI 一致。

重点 PR 1: vfork / clone 语义修复

这个 PR 修的是父进程等待条件，保证 vfork 类子进程在 exec/exit 前不会让父进程提前继续。

问题

- 旧实现把等待条件错误收窄，部分 CLONE_VFORK 场景没有等待。
- 父进程提前返回会破坏 shell、busybox、cargo 依赖的同步顺序。

修复与测例

- 修复 CLONE_VFORK 等待条件，保证父进程等待 vfork_done。
- 测例：test-vfork。
- 影响范围：BusyBox / shell / cargo 子进程创建路径。

成果：修复进程创建 ABI 语义，使 BusyBox、shell 和 cargo 子进程创建路径具备更稳定的行为基础。

背景 2: futex、robust list 与线程退出

pthread/mutex 的稳定性依赖 futex 快慢路径，以及线程异常退出时内核对遗留锁状态的清理能力。

术语定义

futex 是“用户态内存字 + 内核等待队列”的同步机制；robust list 是线程登记给内核的持锁链表，用于退出时修复 owner died 状态。

用户态快路径

互斥锁、条件变量先用原子指令读写内存字；无竞争时不进入内核，这是 futex 高性能的基础。

内核慢路径

竞争失败时 FUTEX_WAIT 睡眠；释放锁时 FUTEX_WAKE 按地址唤醒等待线程，避免无谓轮询。

robust list

线程持锁退出时，内核扫描登记链表，标记 owner died 并唤醒等待者，避免锁永久卡死。

退出路径容错

链表指针来自用户态，可能为空、坏地址或跨页；清理路径必须把它当作不可信输入处理。

futex 正确性取决于用户态内存字与内核等待队列一致，退出清理还必须安全处理异常用户内存。

重点 PR 2: futex 与线程退出清理

这个 PR 的重点是让线程退出路径能够安全处理坏 robust-list entry 和 pending futex。

问题

- 用户态可以传坏 robust-list 指针，内核不能因此拖垮退出路径。
- pending futex 与普通 entry 混在一起，会让 pthread 退出/回收变得脆弱。

修复与测例

- 坏 entry 容错清理，pending 单独处理。
- 测例：test-futex-robust-list。
- 关联：teardown/context usercopy 修复退出路径。

成果：退出清理路径可以安全处理用户态异常输入，避免单个坏 robust-list 破坏进程回收流程。

背景 3: VFS 与构建型 workload 的文件系统压力

构建系统不会只访问单一文件，它会反复组合路径解析、元数据更新、目录遍历、临时文件替换和设备 I/O。

术语定义

VFS 是内核统一文件系统接口层，屏蔽 tmpfs、ext4/rsext4、设备节点、pipe 等后端差异；应用只看到一组统一 syscall。

VFS 抽象层

负责路径解析、dentry/inode/file 对象和权限生命周期，让 open/read/write/rename 等 syscall 有一致语义。

tmpfs：内存文件系统

主要维护内存中的文件对象和目录树；rename/unlink/open/exec 必须保持同一生命周期视图。

ext4 / rsext4

磁盘文件系统需要管理 inode bitmap、block group、目录项和数据块分配；元数据状态会影响后续创建。

Cargo 真实压力

编译会密集创建小文件、原子 rename、目录扫描、unlink，以及 pipe/device I/O；组合 workload 才是真实检验。

构建型 workload 会组合触发路径解析、元数据分配、rename/unlink 和设备 I/O，检验 VFS 后端一致性。

重点 PR 3: 文件系统与 VFS 正确性

这一组 PR 面向真实应用的文件系统访问模式，补齐 inode 分配、rename/exec 和设备传输语义。

#695 rsex4 inode bitmap

ext4 block group 可标记 inode bitmap 未初始化；allocator 需要识别并初始化，再继续分配 inode。

#844 tmpfs rename exec

tmpfs 的目录项和可执行文件生命周期要一致；rename/unlink 不能让后续 exec/open 看到错乱状态。

#800 device transfer

VFS 设备节点读写要遵守完整 transfer 语义，否则 busybox、构建脚本等用户态工具会拿到短读写。

成果：文件系统层面对真实应用的目录项、inode 和设备 I/O 行为更加接近 Linux 预期。

实验 4: StarryOS guest 内自举编译

实验目标是在 StarryOS userland 中运行 cargo build, 完成 StarryOS 自身构建并输出 PASS marker。



- macOS native release 参考: 同 StarryOS kernel 配置, host cargo j1=96s, j8=42s。
- guest 最优: AArch64/HVF 8 核 StarryOS guest, 完整 self-build 331s。
- 判据: 进入 userland、找到 rootfs 内 StarryOS ELF、打印 PASS、无 panic/trap/FATAL/error。

多核编译性能结果

通过脚本和构建参数调节, guest 完整 self-build 从 951s 降到 331s, 约 3 倍, 已稳定进入 400s 内。

951s

最慢 guest baseline

642s

默认 8 核 guest

331s

当前最快 guest

42s

macOS native j8

96s

macOS native j1

端到端真正收益: $951s / 331s = 2.87x$

guest 编译并行: $422s / 341s = 1.24x$

macOS native 参考: $96s / 42s = 2.29x$

链接/串行尾段: $642s / 427s = 1.50x$

注: macOS native release 未设置 opt0/LTO off/CGU256; guest 最优是演示 profile, 二者用于说明量级和瓶颈归属。

结论: 完整 guest self-build 已完成; 并行收益受 Cargo critical path、guest OS/FS/SMP 和串行尾段共同限制。

性能模型与测试细项

每个瓶颈都用接近 Cargo 行为的微基准拆开验证，而不是只给粗略归因。

$$T_{\text{build}}(N) = T_{\text{std/cache}} + T_{\text{serial}} + T_{\text{parallel}}/N + T_{\text{fs}}(N) + T_{\text{smp}}(N) + T_{\text{wait}}(N)$$

进程链路

fork/exec/wait 1000
次: 20/11/5/3s, 8 路
6.67x。模拟 cargo 反复
spawn rustc。

短任务 wave

24 个
build-script: 27/22/27/32
/32s。2
路最快，过并行会被
OS/FS/SMP 成本吃掉。

FS metadata

rsext4 fileio
jobs=8: 47.5s→14.5s。模拟
target 小文件
create/write/rename/unlink
。

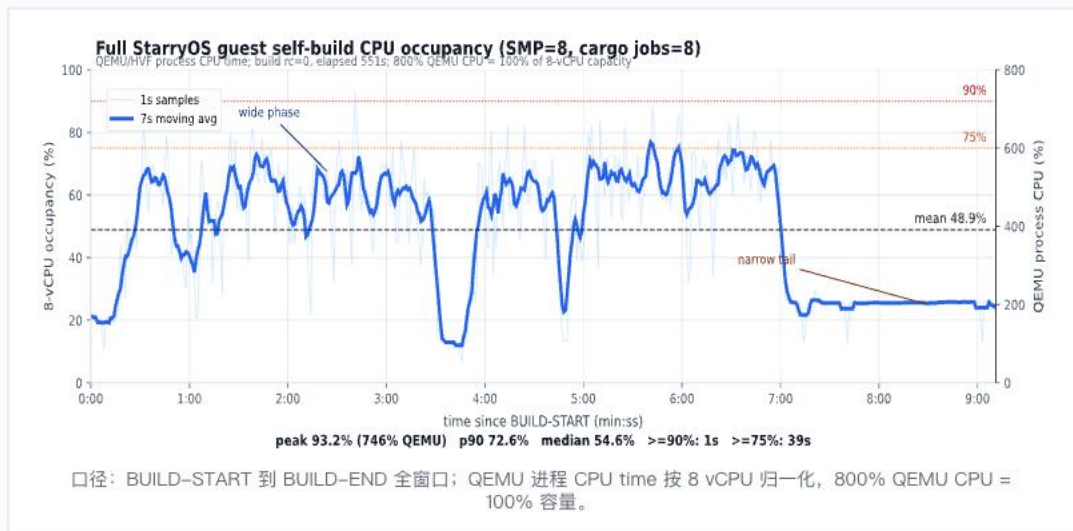
宿主/QEMU参考

macOS native release
96/42=2.29x; guest
jobs-only
422/341=1.24x。raw CPU
MTTCG 3.17x 不作 RISC-V
correctness。

final link timestamp: artifact tail 约 2s, starry-kernel 后段约 17–20s; 331s 版本主瓶颈不是纯链接。

为什么 8 核总是打不满

完整 551s 采样显示：8 核 guest 能短时拉高到 93.2%，但均值只有 48.9%，90% 以上仅 1 秒。



证据等级：曲线/日志直接测得；原因是由 Cargo timing、host 对照和 guest 短基准共同支撑的工程归因。

1. 负载宽度阶段性不足

证据：完整曲线只有 39s >=75%，>=90% 只有 1s。推论：多数时间没有 8 个持续可运行编译任务。

2. rustc/尾段有限并行

证据：约 7:00 之后进入窄尾段，QEMU CPU 长时间约 200%，即约 2 个忙 vCPU。

3. 短任务过并行会反吃

证据：24 个 build-script wave, j1/j2/j4/j6/j8 = 27/22/27/32/32s, 2 路最快。

4. OS/FS 路径确实影响

证据：fork/exec 20/11/5/3s
说明多核有效：rsexrt4 j8 47.5s -> 14.5s 说明 FS 路径曾是瓶颈。

结论：setting 已生效，8 核能被用起来；打不满主要来自 Cargo/Rust 负载宽度、尾段串行化和 guest OS/FS 短链路成本。

对齐对比：为什么 host 有收益，guest 只有 1.24x

macOS native release 证明 Cargo 本身有并行收益；guest jobs-only 更低，说明差距落在 guest OS/FS/SMP 承载成本上。

96s → 42s

macOS native release, 2.29x

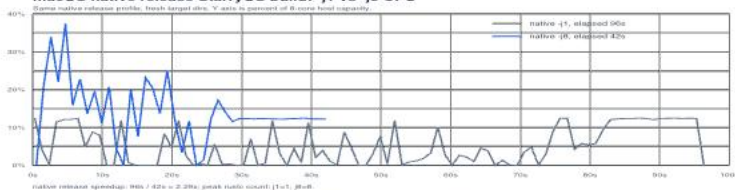
422s → 341s

guest jobs-only, 1.24x

951s → 331s

guest 端到端调优, 2.87x

macOS native release StarryOS build: -j1 vs -j8 CPU



同一 native release profile: -j8 比 -j1 快，但 CPU 仍远低于 8 核常满。

结论

并行空间存在；但完整 cargo 的可运行宽度、短任务成本和串行尾段共同限制收益。guest 还额外承担 OS/FS/SMP 路径成本。

现场讲法：host 是可并行上界；guest 是 OS 真实承载能力；两者差距就是本实验定位和修复的对象。

成因拆解：Cargo、OS、QEMU 分别承担什么

最后把瓶颈按归属拆清楚，避免把所有慢都归因给 QEMU 或 Cargo。

观测	归因	已经做了什么
host 2.29x 但 guest jobs-only 1.24x	Cargo 图有并行空间；差距主要出在 guest 运行成本。	区分 native host 与 guest profile，拆分 syscall、VFS、scheduler、QEMU 边界。
fork/exec/wait 8 路 6.67x	进程创建和等待这条 OS 热路径本身可以扩展。	#842 CPU topology、#926 wakeup progress、#879 RawMutex handoff。
build-script wave 2 路最快，8 路变慢	Cargo 短任务过并行后，被 pipe/wait、文件 metadata 和锁竞争反吃。	用短基准定位 T_wait(N)、T_fs(N)、T_smp(N)，避免只等完整 build。
final link tail 约 2s	331s 版本主瓶颈不是纯链接器，而是 build graph 和 codegen 尾段。	关闭 LTO、opt0、cgu256，串行尾段 642s -> 427s。

结论：self-build 已跑通；多核不是无效，剩余瓶颈已拆成 Cargo critical path、OS 短任务成本和虚拟化边界三类。

总结：真实 workload 驱动的 StarryOS 改进

主线是让 StarryOS 在 guest 内编译 StarryOS，并把失败路径拆成可验证、可合入、可复现的工程结果。

Self-build 跑通

AArch64/HVF 8 核 guest 内完成
StarryOS self-build。

最好结果 331s；端到端从 951s 调到
331s。

OS 修复合入

12 个 PR 已进入 TGOSKit dev。

覆盖 vfork/futex、VFS/device、SMP
调度、usercopy
上下文等真实系统语义。

性能边界拆清

fork/exec/wait 可到 6.67x，macOS
native 编译 2.29x，guest jobs-only
1.24x。

瓶颈来自 Cargo critical path、OS
小任务成本和虚拟化边界。

最终结果：一套可复现 self-build demo，一组已合入 OS 修复，一份可解释的多核性能证据。